Session 5 - Dynamic parallelism and asynchronous operation
Geavanceerde computerarchitectuur: labosessie

(B-KUL-JLI1QS)

Elias **Neels**, Wout Batselé

March 26, 2026



Year: 2026

Class: MELEC

Instructor:

Contents

1 Context **2**

2 Methodology **2**

3 Observations/results **2**
3.1 Nsight Systems 3
3.1.1 Uitleg hoe ik nsight werkende heb gekregen 3
3.2 Analyse van de Nsight Systems Tijdlijn 4
3.2.1 Synchrone Uitvoering (Default Stream) 4
3.2.2 Asynchrone Uitvoering (Meerdere Streams) 4

4 Conclusion **5**

A Code **5**

1 Context

In deze sessie behandelen we technieken om de efficiëntie van GPU-programma's te verhogen door het gebruik van asynchrone operaties. Hiervoor baseren we ons op code die we eerder schreven voor sessie 2. Deze passen we lichtelijk aan (implementatie som, product, minimum en maximum operaties). Deze operaties worden vervolgens synchroon en asynchroon uitgevoerd om vervolgens de performantie winst van de respectievelijke uitvoertechnieken te evalueren.

2 Methodology

De basis van onze implementatie is de reductie-kernel uit de tweede labosessie, die is aangepast om via een parameter verschillende wiskundige operaties uit te voeren.

- **Synchrone Workflow**: De CPU bereidt de data voor, kopieert deze naar de GPU, en start de kernel. Pas nadat de GPU klaar is (`cudaDeviceSynchronize`), start de CPU met het voorbereiden van de data voor de volgende operatie.

- **Asynchrone Workflow**: We maken gebruik van `cudaMemcpyAsync` om te zorgen dat de CPU niet geblokkeerd wordt tijdens de GPU-berekeningen. Hierdoor kan de CPU de array voor de volgende operatie al generen terwijl de GPU nog bezig is met de vorige operatie.

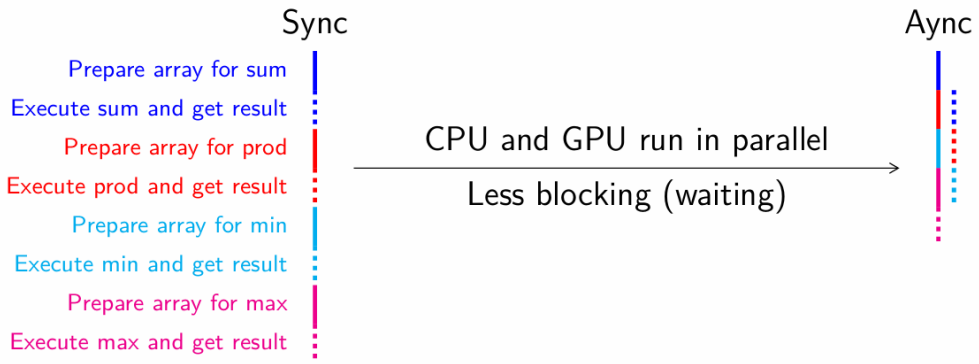
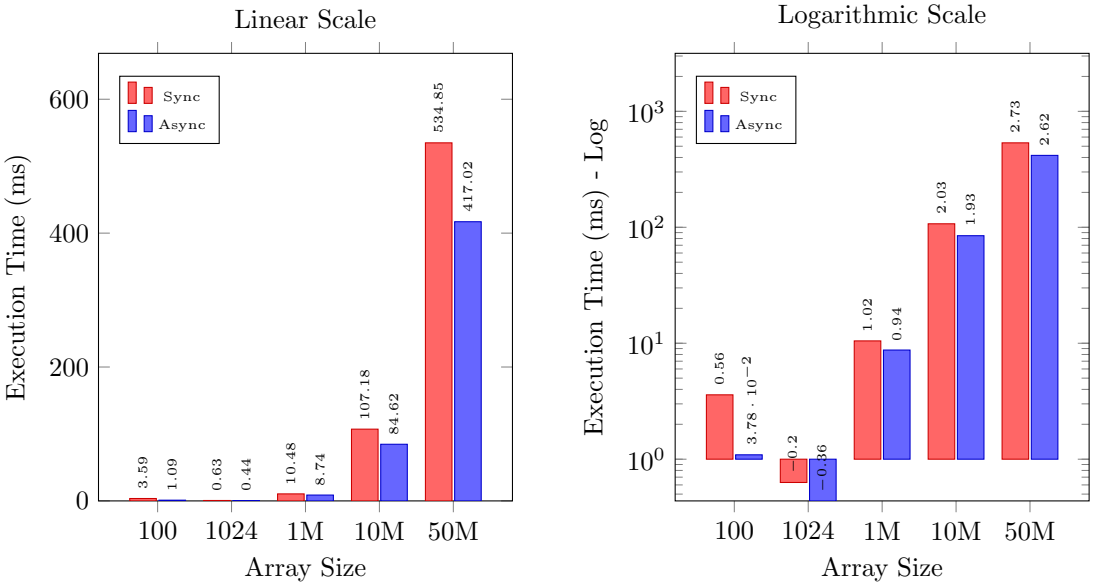


Figure 1: Synchroon vs Asynchroon

3 Observations/results

De tests laten zien dat de asynchrone workflow aanzienlijk efficiënter is. In de synchrone modus zien we veel 'blocking' momenten waarbij hardware-resources onbenut blijven omdat de CPU en GPU op elkaar wachten. Hierdoor is de totale uitvoertijd aanzienlijk langer.
Bij de asynchrone uitvoering overlappen de taken. De CPU is al bezig met het prepareren van het volgende blok data terwijl de GPU-threads de reductie uitvoeren. Dit resulteert in een kortere totale uitvoeringstijd.



(a) Linear y-axis

(b) Logarithmic y-axis

Figure 2: Resultaten gemiddelde execution time (gemiddelde berekend door de code 10x te runnen en handmatig het gemiddelde te berekenen)

3.1 Nsight Systems

3.1.1 Uitleg hoe ik nsight werkende heb gekregen

Om de asynchrone executie en het gedrag van de CUDA-streams te analyseren, is NVIDIA Nsight Systems gebruikt. Omdat de code binnen het Windows Subsystem for Linux (WSL) werd ontwikkeld, is de volgende cross-platform werkwijze gehanteerd:

a. **Installatie op beide systemen:** Installeer NVIDIA Nsight Systems zowel op de Windows-host (voor de grafische interface) als binnen de WSL-omgeving (de command-line tool `nsys` voor het profilen).

b. **Compilatie van de broncode:** Compileer de `.cu` code in de WSL-terminal naar een .exe filE. De `-lineinfo` vlag toegevoegd om de hardware-executie te kunnen linken aan de specifieke regels code

```
nvcc mijn_code.cu -o mijn_programma.exe -lineinfo
```

c. **Genereren van het profileringsrapport:** Voer het programma uit via de Nsight profiler in de WSL-terminal. Tracing voor CUDA (`-t cuda`) wordt geactiveerd en de output wordt weggeschreven naar een rapportbestand:

```
nsys profile -t cuda -o name_report ./mijn_programma
```

Dit commando genereert na uitvoering een bestand genaamd `name_report.nsys-rep` in de huidige Linux-directory.

d. **Visualisatie in Windows:** Open de NVIDIA Nsight Systems applicatie op Windows. Ga naar *File* > *Open* en navigeer naar het gegenereerde `name_report.nsys-rep` bestand om de tijdlijn en de overlappende streams visueel te analyseren.

3.2 Analyse van de Nsight Systems Tijdlijn

Om de theoretische prestatiewinst van asynchrone executie te verifiëren, is het programma geanalyseerd met NVIDIA Nsight Systems. Figuur 3 toont de hardware-tijdlijn (`CUDA HW`) van zowel de synchrone als de asynchrone implementatie.

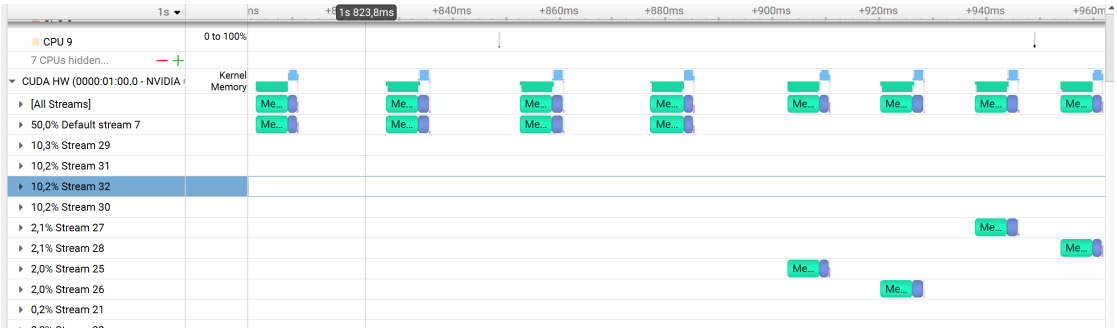


Figure 3: Nsight Systems tijdlijn. De rij `Default stream 7` toont de synchrone uitvoering (links). De genummerde rijen (`Stream 25` t/m `28`) tonen de asynchrone uitvoering (rechts) met het kenmerkende trapsgewijze patroon.

De geprofileerde tijdlijn kan worden opgesplitst in twee duidelijk te onderscheiden fases:

3.2.1 Synchrone Uitvoering (Default Stream)

In de linkerhelft van de tijdlijn worden alle operaties uitgevoerd op `Default stream 7`. Deze blokken staan strikt sequentieel achter elkaar gepositioneerd. Dit bevestigt het blokkerende karakter van synchrone executie: de CPU moet wachten op het antwoord van de GPU voordat de volgende datatransfer of kernel gelanceerd kan worden. Hierdoor lopen de CPU en GPU niet parallel, wat resulteert in een suboptimale benutting van de beschikbare rekencapaciteit.

3.2.2 Asynchrone Uitvoering (Meerdere Streams)

In de rechterhelft van de tijdlijn is de asynchrone implementatie zichtbaar. In plaats van één enkele stream, zijn de operaties nu verdeeld over afzonderlijke streams (`Stream 25`, `26`, `27` en `28`).
Dit illustreert dat de CPU en GPU parallel werken. Terwijl de GPU bijvoorbeeld data verwerkt in `Stream 25`, is de CPU niet geblokkeerd en kan deze tegelijkertijd al de geheugentransfers en kernel-launches voor de volgende streams inplannen.
Daarnaast is er tussen de opeenvolgende asynchrone streams ook steeds een idle time zichtbaar. Een mogelijke verklaring hiervoor is dat dit de tijd vertegenwoordigt die de CPU nodig heeft om de data voor de volgende array te genereren en de overhead van het communiceren over de PCIe-bus. Ondanks deze minimale overhead zorgt de compactere spreiding van de asynchrone blokken dat de totale executietijd is gereduceerd ten opzichte van de synchrone methode.

4 Conclusion

Aan de hand van de resultaten kunnen we zien dat het asynchroon maken van de kernels veel voordeel oplevert. Dit toont op zijn beurt aan dat dat parallellisme op de GPU verder gaat dan alleen het verdelen van threads. Door de berekening te overlappen via asynchrone kernels wordt het volledige potentieel van de GPU gebruikt wat prestaties drastisch verbeterd.

A Code

```

#include <iostream>
#include <vector>
#include <chrono>
#include <cmath>
#include <cuda_runtime.h>

#define BLOCK_SIZE 1024

// Enum to define our 4 operations
enum Operation { SUM = 0, PROD = 1, MIN = 2, MAX = 3 };

// --- Device Helper Functions ---
template<int OP>
__device__ float apply_op(float a, float b) {
    if (OP == SUM) return a + b;
    if (OP == PROD) return a * b;
    if (OP == MIN) return fminf(a, b);
    if (OP == MAX) return fmaxf(a, b);
    return 0.0f;
}

template<int OP>
__device__ float get_init_val() {
    if (OP == SUM) return 0.0f;
    if (OP == PROD) return 1.0f;
    if (OP == MIN) return 1e30f;  // Infinity-ish
    if (OP == MAX) return -1e30f; // -Infinity-ish
    return 0.0f;
}

// --- Generic Shared Memory Reduction Kernel ---
template<int OP>
__global__ void reduce_kernel(float *data, int n, float *block_results) {
    extern __shared__ float sdata[];

    int tid = threadIdx.x;
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    // Load data into shared memory with boundary checks
    sdata[tid] = (idx < n) ? data[idx] : get_init_val<OP>();
    __syncthreads();

    // Reduction in shared memory
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] = apply_op<OP>(sdata[tid], sdata[tid + s]);
        }
        __syncthreads();
    }
```

```
51
52      // Write the block's result to global memory
53      if (tid == 0) {
54          block_results[blockIdx.x] = sdata[0];
55      }
56  }
57
58  // --- CPU Data Generation ---
59  // Simulates work on the CPU without the massive bottleneck of rand()
60  void generate_data(float* data, int n, int op) {
61      for (int i = 0; i < n; ++i) {
62          if (op == PROD) {
63              // Values in (0, 1] to avoid overflow/underflow
64              // Using a simple modulo instead of rand() for speed
65              data[i] = 1.0f - (i % 1000) / 50000.0f;
66          } else {
67              data[i] = (i % 100) / 10.0f; // 0.0 to 9.9
68          }
69      }
70  }
71
72  // Finish the reduction on the CPU (for the block results)
73  float finalize_reduction_cpu(float* block_results, int num_blocks, int op) {
74      float result = (op == SUM) ? 0.0f : (op == PROD) ? 1.0f : (op == MIN) ? 1e30f
        : -1e30f;
75      for (int i = 0; i < num_blocks; i++) {
76          if (op == SUM) result += block_results[i];
77          else if (op == PROD) result *= block_results[i];
78          else if (op == MIN) result = fminf(result, block_results[i]);
79          else if (op == MAX) result = fmaxf(result, block_results[i]);
80      }
81      return result;
82  }
83
84  // --- Main Execution Workflow ---
85  void run_comparison(int n) {
86      std::cout <<  \n--- Array size:   << n <<   ---  << std::endl;
87
88      int gridSize = (n + BLOCK_SIZE - 1) / BLOCK_SIZE;
89      int sharedMemSize = BLOCK_SIZE * sizeof(float);
90
91      // Allocate Pinned Host Memory (required for true Async Memcpy behavior)
92      float *h_data[4], *h_block_results[4];
93      float *d_data[4], *d_block_results[4];
94      cudaStream_t streams[4];
95
96      for (int i = 0; i < 4; i++) {
97          cudaMallocHost(&h_data[i], n * sizeof(float)); // Pinned memory
98          cudaMallocHost(&h_block_results[i], gridSize * sizeof(float));
99          cudaMalloc(&d_data[i], n * sizeof(float));
100         cudaMalloc(&d_block_results[i], gridSize * sizeof(float));
101         cudaStreamCreate(&streams[i]);
102     }
103
104     // ========================================
105     // APPROACH 1: SYNCHRONOUS
106     // ========================================
107     auto start_sync = std::chrono::high_resolution_clock::now();
108
109     for (int i = 0; i < 4; i++) {
110         // 1. CPU Work
111         generate_data(h_data[i], n, i);
```

```
112
113         // 2. Synchronous Copy to GPU
114         cudaMemcpy(d_data[i], h_data[i], n * sizeof(float), cudaMemcpyHostToDevice
    );
115
116         // 3. Kernel Execution
117         if (i == SUM) reduce_kernel<SUM><<<gridSize, BLOCK_SIZE, sharedMemSize>>>(
    d_data[i], n, d_block_results[i]);
118         if (i == PROD) reduce_kernel<PROD><<<gridSize, BLOCK_SIZE, sharedMemSize
    >>>(d_data[i], n, d_block_results[i]);
119         if (i == MIN) reduce_kernel<MIN><<<gridSize, BLOCK_SIZE, sharedMemSize>>>(
    d_data[i], n, d_block_results[i]);
120         if (i == MAX) reduce_kernel<MAX><<<gridSize, BLOCK_SIZE, sharedMemSize>>>(
    d_data[i], n, d_block_results[i]);
121
122         // 4. Synchronous Copy back to CPU
123         cudaMemcpy(h_block_results[i], d_block_results[i], gridSize * sizeof(float
    ), cudaMemcpyDeviceToHost);
124
125         // 5. Finalize CPU
126         finalize_reduction_cpu(h_block_results[i], gridSize, i);
127     }
128
129     cudaDeviceSynchronize(); // Ensure everything is finished
130     auto end_sync = std::chrono::high_resolution_clock::now();
131     std::chrono::duration<double, std::milli> time_sync = end_sync - start_sync;
132     std::cout <<  Synchronous Total Time:    << time_sync.count() <<   ms\n ;
133
134     // ========================================
135     // APPROACH 2: ASYNCHRONOUS
136     // ========================================
137     auto start_async = std::chrono::high_resolution_clock::now();
138
139     // 1. Prepare the FIRST array before entering the loop
140     // This gives the GPU something to do immediately when the loop starts
141     generate_data(h_data[0], n, 0);
142
143     for (int i = 0; i < 4; i++) {
144         // 2. Asynchronous Copy to GPU (Non-blocking)
145         cudaMemcpyAsync(d_data[i], h_data[i], n * sizeof(float),
    cudaMemcpyHostToDevice, streams[i]);
146
147         // 3. Asynchronous Kernel Execution (Non-blocking)
148         if (i == SUM) reduce_kernel<SUM><<<gridSize, BLOCK_SIZE, sharedMemSize,
    streams[i]>>>(d_data[i], n, d_block_results[i]);
149         if (i == PROD) reduce_kernel<PROD><<<gridSize, BLOCK_SIZE, sharedMemSize,
    streams[i]>>>(d_data[i], n, d_block_results[i]);
150         if (i == MIN) reduce_kernel<MIN><<<gridSize, BLOCK_SIZE, sharedMemSize,
    streams[i]>>>(d_data[i], n, d_block_results[i]);
151         if (i == MAX) reduce_kernel<MAX><<<gridSize, BLOCK_SIZE, sharedMemSize,
    streams[i]>>>(d_data[i], n, d_block_results[i]);
152
153         // 4. Asynchronous Copy back to CPU (Non-blocking)
154         cudaMemcpyAsync(h_block_results[i], d_block_results[i], gridSize * sizeof(
    float), cudaMemcpyDeviceToHost, streams[i]);
155
156         // 5. PREPARE THE NEXT ARRAY!
157         // The CPU executes this immediately while the GPU is processing the lines
     above.
158         // This is where the actual time-saving overlap happens.
159         if (i + 1 < 4) {
160             generate_data(h_data[i+1], n, i + 1);
```

```
        }
    }

    // Wait for all streams to finish before doing the final CPU reduction
    cudaDeviceSynchronize();

    // Finalize CPU (done after sync to ensure data is ready)
    for (int i = 0; i < 4; i++) {
        finalize_reduction_cpu(h_block_results[i], gridSize, i);
    }

    auto end_async = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> time_async = end_async - start_async
    ;
    std::cout <<  Asynchronous Total Time:   << time_async.count() <<   ms\n ;

    // --- Cleanup ---
    for (int i = 0; i < 4; i++) {
        cudaFreeHost(h_data[i]);
        cudaFreeHost(h_block_results[i]);
        cudaFree(d_data[i]);
        cudaFree(d_block_results[i]);
        cudaStreamDestroy(streams[i]);
    }
}

int main() {
    srand(time(NULL));

    // Warm up the GPU
    cudaFree(0);

    // Process different array sizes
    run_comparison(100);
    run_comparison(1024);
    run_comparison(1000000);
    run_comparison(10000000);
    run_comparison(50000000); // Larger arrays demonstrate greater async benefits

    return 0;
}
```

Listing 1: CUDA kernels for exercise parallelism and asynchrnous operation

References

[1] Feys T. Gent (2026) *session-5*, Lab notes.